

Instituto Brasileiro de Ensino, Desenvolvimento e Pesquisa — IDP

Curso de Ciência da Computação

**A Teoria dos Conjuntos e Diagramas de Venn aplicados à
Modelagem de Dados:
Fundamentos Lógicos para Sistemas Relacionais**

Marco Antônio Rodrigues Siqueira

Gael Ceregatti Murad

Davi Brandão

Carlos Marquim

Lucas Viana

Rafael Dias

Brasília, 2026

RESUMO

Na contemporaneidade, a relevância das empresas é diretamente proporcional ao gerenciamento dos dados referentes às suas operações. Adjacente à popularização da ciência de dados, linguagens voltadas à estruturação, manutenção e visualização de dados, como o SQL, foram desenvolvidas com o propósito de democratizar o acesso às vantagens proporcionadas pela análise de grandes volumes de dados. Entretanto, o foco restrito à parte prática da análise em detrimento do ferramental teórico, fundamentado na Teoria dos Conjuntos e na Álgebra Relacional, gera, até os dias de hoje, problemas envolvendo redundâncias em bancos de dados extensos, comprometendo, portanto, a veracidade e a acurácia das informações extraídas a partir de sua coleta e análise. Este trabalho analisa as conexões formais entre o Modelo Relacional e a Teoria dos Conjuntos, e propõe a validação lógica por meio do uso de Diagramas de Venn, por serem inteligíveis tanto para leigos quanto para profissionais, e por sua capacidade de mapear todas as regiões e combinações possíveis de uma operação, constituindo uma eficaz ferramenta de revisão humana e verificação rigorosa que garante a integridade das consultas e da estrutura lógica dos sistemas relacionais.

Palavras-chave: Banco de Dados Relacional; SQL; Teoria dos Conjuntos; Álgebra Relacional; Diagramas de Venn; Redundância de Dados.

1 INTRODUÇÃO

A hiperconectividade e a digitalização são pilares fulcrais e preponderantes no mercado de trabalho hodierno (STONEBRAKER; HELLERSTEIN, 2005). Percebe-se que, quanto mais digitalizado, maior a exigência por sistemas capazes de armazenar, processar e avaliar dados (CHAMBERLIN, 2012), com o mote de que estes sejam utilizados como ativos estratégicos para otimizar vendas, administrar negócios e organizar as dinâmicas laborais (DATE, 2012). Essa estruturação corrobora a precisão em sistemas de suporte à decisão, característica que (CODD, 1970) é determinante para o desempenho de uma empresa no século XXI.

Atualmente, a literatura moderna indica que organizações com uma estrutura sólida de armazenamento e análise de dados são mais capacitadas para alocar recursos em áreas estratégicas e mobilizar modelos de negócio inovadores, atingindo uma performance satisfatória e livre de inconsistências (CODD, 1970; STONEBRAKER; HELLERSTEIN, 2005). Neste escopo, a vasta gama de possibilidades competitivas ressalta a importância de adquirir um sistema de banco de dados que seja não apenas robusto em capacidade, mas também intuitivo para que profissionais de diferentes setores possam manipular informações conforme as demandas do negócio (DATE, 2012; BEAULIEU, 2020). Assim, faz-se imprescindível salientar que os sistemas relacionais são a "pedra-mestra" nesse tipo de operação (CODD, 1970), por se consolidarem como arquiteturas que utilizam linguagens mais inteligíveis (CHAMBERLIN, 2012), permitindo que a complexidade da álgebra de dados seja traduzida em comandos intuitivos para o usuário final (ATWOOD, 2007; BEAULIEU, 2020).

O SQL (Structured Query Language) é a linguagem padrão de consulta a bancos de dados relacionais. Sua relação com a Teoria dos Conjuntos é mediada pela Álgebra Relacional: cada comando SQL implementa uma operação de conjuntos sobre relações. No entanto, Chamberlin, um dos criadores do SQL, documenta que a linguagem foi projetada para ser acessível a usuários não matemáticos, fazendo com que o SQL seja operacionalmente acessível, mas tornou menos explícita a semântica formal que determina o comportamento de cada operação. Beaulieu exemplifica esse distanciamento ao apresentar, no livro *Learning*

SQL, 2nd edition, consultas SQL como construções sintáticas sem explicitar que são operações de conjuntos com propriedades matemáticas.

O resultado é evidente: profissionais de banco de dados executam operações cujas propriedades matemáticas não dominam completamente, operando sem o ferramental teórico que lhes permitiria raciocinar de forma plena sobre os resultados de suas consultas, originando, conseqüentemente, inconsistências. Sob um prisma paradoxal, poucos investimentos em estruturas de análise e lacunas na padronização da coleta de registros corroboram limitações na formulação de soluções estratégicas (IBM) para empresas.

Nesse diapasão, a existência de anomalias nos dados, redundâncias, inconsistência de dados em arquivos diferentes e informações ambíguas ou mal definidas (Data Smells, 2022), frutos da ausência parcial dos fundamentos da Teoria dos Conjuntos e das Formas Normais durante a etapa de modelagem lógica (IBM), dificulta a realização de consultas precisas e a extração de insights confiáveis.

Atwood demonstrou visualmente que cada tipo de JOIN em SQL corresponde a uma região específica do Diagrama de Venn de duas tabelas. Essa contribuição permanece na literatura técnica como recurso didático sem a formalização matemática que lhe conferiria status de instrumento de verificação.

Neste viés, este trabalho busca validar o uso de instrumentos lógicos, como Diagramas de Venn e de Euler, como mecanismo que garanta a integridade e a acurácia das informações reveladas por dados, passando a complementar, portanto, as atividades das empresas que utilizam sistemas relacionais. Por serem visualmente inteligíveis, esses diagramas tornam conceitos de Álgebra Relacional mais evidentes para a revisão humana, tanto para um leigo quanto para um profissional na área, comprovando a necessidade de estudos que aprofundem o uso de Diagramas de Venn como ferramenta para compreender a Teoria dos Conjuntos e validar a estrutura lógica de sistemas relacionais.

2 REVISÃO CONCEITUAL

2.1 Conjuntos

De acordo com Antonio Alfredo Ferreira Loureiro, a Teoria dos Conjuntos é o ramo da matemática que estuda as propriedades e as relações entre coleções de objetos. Um

conjunto é entendido como uma coleção de objetos bem definidos, que podem ser qualquer coisa: números, letras, pessoas ou até outros conjuntos, e são denominados elementos.

A relação de pertencimento é o conceito fundamental dessa teoria. Segundo Jaime Campos Ferreira, se um objeto x é um dos elementos que compõem o conjunto A , dizemos que x pertence a A , utilizando o símbolo $\in$ ($x \in A$). Caso o objeto não faça parte da coleção, utiliza-se o símbolo $\notin$ ($x \notin A$).

Ferreira reforça que a relação de pertencimento é a base para definir outras relações, como a de inclusão (quando um conjunto está contido em outro).

Na prática, se definirmos um conjunto A como os números pares menores que 10, temos $A = \{2, 4, 6, 8\}$.

- Dizemos que $2 \in A$ (2 pertence a A), pois é um elemento do conjunto.
- Dizemos que $3 \notin A$ (3 não pertence a A), pois não é um elemento do conjunto.

Essa distinção clara entre o que está "dentro" e o que está "fora" do conjunto, conforme as regras de lógica matemática detalhadas por Ferreira, é o que permite operações como a seleção, a filtragem e a validação de dados em bancos de dados.

A necessidade de definir conjuntos e pertencimento reside na fundamentação de toda a matemática moderna. Joan Bagaria explica que a Teoria dos Conjuntos serve como a linguagem comum e a fundação sobre a qual quase todos os objetos matemáticos (como números, funções e estruturas geométricas) são formalmente definidos. Além disso, Loureiro destaca que essa estrutura permite a organização de informações e a visualização de relações complexas através de métodos gráficos, como os Diagramas de Venn, facilitando a compreensão de interseções e uniões entre diferentes grupos.

A definição moderna de conjunto vai além da simples intuição. Como aponta Bagaria, para evitar contradições lógicas, a matemática utiliza axiomas (regras fundamentais). Um dos mais importantes é o Axioma da Extensionalidade, que afirma que um conjunto é determinado unicamente pelos seus elementos, ou seja, a ordem em que os elementos são apresentados ou a repetição deles não altera a identidade do conjunto.

2.2 Diagramas de Venn

Venn (1880) propõe os diagramas como um instrumento de verificação lógica das relações entre conjuntos, permitindo testar a validade de proposições de forma sistemática e rigorosa.

No escopo do uso de diagramas como instrumentos de verificação das relações entre conjuntos, faz-se imprescindível salientar a diferença entre Diagramas de Venn e Diagramas de Euler. Moktefi e Lemanski (2022) formalizam a propriedade que os distingue: os Diagramas de Venn representam todas as regiões possíveis resultantes da combinação de conjuntos, inclusive as regiões vazias, as quais são marcadas por sombreamento. Por sua vez, os Diagramas de Euler representam apenas as relações efetivamente existentes. Um diagrama que não representa áreas sem elementos não permite verificar se uma operação retorna um conjunto vazio, o que é uma das verificações necessárias para avaliar a precisão dos resultados em consultas SQL.

A título de exemplificação, um Diagrama de Venn de dois conjuntos A e B possui exatamente quatro regiões: $A \setminus B$ (exclusiva de A), $A \cap B$ (termos em comum ou superposição), $B \setminus A$ (exclusiva de B) e $(A \cup B)^c$ (o complemento da união entre A e B, ou seja, região externa aos conjuntos A e B). Cada operação fundamental corresponde a uma combinação específica dessas regiões, o que torna o diagrama um mapa completo das possíveis relações entre os dois conjuntos.

2.3 Álgebra Relacional

O Modelo Relacional, proposto por Edgar F. Codd em 1970 no artigo *A Relational Model of Data for Large Shared Data Banks*, conecta os fundamentos da Teoria dos Conjuntos ao domínio computacional de dados. Na seção 1.1 do artigo, Codd define: dada uma coleção de domínios $D_1, D_2, \dots, D_n$, uma relação R sobre esses domínios é um subconjunto do produto cartesiano $D_1 \times D_2 \times \dots \times D_n$. Nesse modelo, cada relação é composta por atributos (colunas) e tuplas (linhas), cujos valores pertencem a domínios específicos. Essa é a mesma definição matemática de relação formulada por Enderton (1977). Na seção 2.1, Codd define as operações relacionais, como seleção, projeção, união, interseção e diferença, como operações de conjuntos sobre os subconjuntos do produto cartesiano.

Dentre essas operações, a seleção é responsável pelo filtro de tuplas (linhas), enquanto a projeção seleciona atributos específicos. A título de exemplificação, em um conjunto de dados contendo os atributos nome e nota, a seleção poderia ser utilizada para retornar apenas

as tuplas cujas notas sejam superiores a cinco. Em contrapartida, a projeção seletora apenas atributos específicos, como a coluna correspondente aos nomes. Sob esse prisma, evidencia-se que o Modelo Relacional consolida a relação entre a Teoria dos Conjuntos e o domínio de dados.

Sob o diapasão das inovações que Codd introduziu, a exigência de que a união de duas relações só é definida se elas possuírem o mesmo número de atributos com domínios correspondentes compatíveis, pois apenas assim o resultado constitui uma relação válida com esquema consistente, configura uma decisão de modelagem, e não uma propriedade da Teoria dos Conjuntos, demonstrando que a transposição de Codd foi deliberada e tecnicamente informada. Stonebraker e Hellerstein (2005) documentam que a vantagem do Modelo Relacional sobre os sistemas hierárquicos se deveu à sua base conjuntista rigorosa. Sistemas que careciam de fundamentos matemáticos bem definidos não conseguiam garantir que a mesma consulta produzisse sempre o mesmo resultado, nem que operações equivalentes fossem reconhecidas como idênticas.

Silberschatz, Korth e Sudarshan (2019) descrevem a Álgebra Relacional como extensão da Teoria dos Conjuntos e estabelecem a correspondência entre operações relacionais e SQL. Date (2004) descreve cada operação da Álgebra Relacional como operação de conjuntos, incluindo a união, interseção, diferença, produto cartesiano, seleção e projeção. Heuser (2009) identifica que erros recorrentes em projetos de banco de dados têm origem na incompreensão das relações lógicas entre os conjuntos de dados que as tabelas representam.

2.4 Banco de Dados

Um banco de dados é uma coleção organizada de dados persistentes que atende a múltiplos usuários e aplicações. O modelo de banco de dados relacional fundamenta-se na formalização proposta por Codd, segundo a qual os dados devem ser organizados em relações compostas por tuplas e atributos (CODD, 1970). No modelo relacional, os dados são organizados em tabelas com esquema definido, sendo pouco flexível, mas muito organizado, íntegro e de fácil consulta. Infere-se, portanto, que sua estrutura não é aleatória, mas planejada de acordo com as necessidades de armazenamento, organização e recuperação das informações.

Em um banco de dados relacional, uma tabela não constitui apenas um mecanismo prático de armazenamento, mas uma representação formal de um conjunto matemático, em

que cada tupla corresponde a um elemento pertencente a um domínio definido. A adoção desse modelo está associada à necessidade de reduzir redundâncias e inconsistências, problemas recorrentes em sistemas que não utilizam uma estrutura lógica rigorosa (HEUSER, 2009). Conforme observado por Rahm e Do, a ausência de organização adequada dos dados implica em custos elevados de correção e manutenção, comprometendo a confiabilidade das informações (RAHM; DO, 2000). Do ponto de vista teórico, o modelo relacional é uma aplicação direta da teoria dos conjuntos e da lógica de predicados, sendo essas bases essenciais para a consistência estrutural dos dados (DATE, 2012). A operacionalização desses conceitos ocorre por meio da linguagem SQL, desenvolvida no contexto do System R, que traduz operações matemáticas em comandos declarativos acessíveis ao usuário (CHAMBERLIN, 2012).

3 DESENVOLVIMENTO

3.1 Matemática e Diagramas de Venn

Um cadastro de clientes de uma empresa pode ser considerado um banco de dados relacional. Nele, uma coluna indica o código do cliente, outra indica o seu nome e uma terceira indica sua idade, por exemplo. Por mais simples que essa estrutura de dados relacional possa ser, para que não haja irregularidade nos dados contidos nela, diversos axiomas devem ser respeitados durante seu preenchimento.

No campo da matemática, muito antes de existirem bancos de dados como o MySQL ou a Cassandra, a Teoria dos Conjuntos era o ramo responsável por estudar as propriedades e as relações entre coleções de objetos. Nesse escopo, um conjunto é entendido como uma coleção de objetos bem definidos, que podem ser qualquer coisa: números, letras, pessoas ou até outros conjuntos, e são denominados elementos (LOUREIRO, [20--]). Tomando como exemplo a situação discorrida previamente, tem-se a seguinte formalização:

$$\text{Clientes} \subseteq \text{dom}(\text{ID}) \times \text{dom}(\text{Nome}) \times \text{dom}(\text{Idade})$$

$$\text{dom}(\text{ID}) = \mathbb{Z}^+$$

$$\text{dom}(\text{Nome}) = \text{String}$$

$$\text{dom}(\text{Idade}) = \mathbb{Z}^+$$

ID (PK)	Nome	Idade
1	Ana	16
2	Bruno	23
3	Carlos	50

Tabela 1 – Cadastro de clientes, Filial A

Polindo ainda mais o exemplo anterior, suponha que possuamos duas tabelas de clientes que contenham os dados da mesma empresa, porém em filiais diferentes, consequentemente contendo dados de clientes diferentes.

ID (PK)	Nome	Idade
4	Antunes	7
2	Bruno	23
5	Cecilia	84

Tabela 2 – Cadastro de clientes, Filial B

Caso queiramos juntar ambas as tabelas, a fim de criar uma base de dados unificada, podemos utilizar a operação de UNIÃO, criando uma tabela nova que contém os dados das duas tabelas originais:

$$A \cup B = \{x \mid x \in A \vee x \in B\} \quad (1)$$

ID (PK)	Nome	Idade
1	Ana	16
2	Bruno	23
3	Carlos	50
4	Antunes	7
5	Cecilia	84

Tabela 3 – Resultado de $A \cup B$, com eliminação automática de duplicatas (Bruno, ID 2)

Vale ressaltar que um fenômeno fulcral ocorre durante essa operação: a exclusão de duplicatas. Note que em ambas as tabelas existe um mesmo cadastro de um cliente (Bruno, ID 2). Durante a união, elementos repetidos são tratados como uma única ocorrência. Esta é a primeira regra da Álgebra Relacional (baseada na Teoria dos Conjuntos) apresentada neste

artigo, e o seu impacto torna-se evidente durante a modelagem de bancos de dados mais avançados.

Agora, suponha que as duas tabelas que desejamos unir sejam as seguintes:

ID (PK)	Nome	Idade
1	Carlos	12
2	João	9
3	Amanda	12

Tabela 4a – Clientes, origem A

ID (PK)	Nome	Idade
1	Amanda	7
1	Carlos	12
2	João	9
3	Amanda	12

Tabela 4b – Resultado inválido: IDs 1 duplicado para pessoas distintas (destacado em amarelo)

Isso resultaria em um erro clássico, baseado em um dos axiomas da Álgebra Relacional, fundamentada na Teoria dos Conjuntos, que diz que, por mais que o domínio das relações seja compatível, a existência de pessoas distintas com o mesmo ID configura uma infração do axioma da Chave Primária.

A Chave Primária consiste em um ou vários atributos cujo valor deve ser obrigatoriamente único para cada tupla de uma relação, atuando como um identificador que impede a duplicata de dados e garante a integridade referencial do modelo (HEUSER, 2009).

Conclui-se, portanto, que a modelagem bem-sucedida de bancos de dados relacionais, por mais simples que seja, exige o conhecimento matemático acerca da Teoria dos Conjuntos e suas propriedades e pode ser simplificada por meio da ilustração de um Diagrama de Venn.

3.2 Álgebra Relacional

O Modelo Relacional, proposto por Edgar F. Codd, conecta os fundamentos da Teoria dos Conjuntos ao domínio computacional de dados (CODD, 1970). Nesse modelo, cada

relação é composta por atributos (colunas) e tuplas (linhas), cujos valores pertencem a domínios específicos. Antes de aplicar, na prática, a Teoria dos Conjuntos no contexto computacional, ou seja, a Álgebra Relacional, faz-se imprescindível compreender as estruturas de dados comuns que empresas utilizam para armazenar informações importantes.

Uma tabela representa logicamente uma relação R , na qual R é um subconjunto do produto cartesiano entre os domínios dos atributos. Visualmente, cada atributo é representado por uma coluna cujo nome também é apresentado no cabeçalho (HEUSER, 2009).

Com isso em mente, no mercado de trabalho, funcionários utilizam operações da Álgebra Relacional a fim de obter uma melhor visualização dos dados. No cenário apresentado previamente, podemos aplicar filtros que mostrem apenas pessoas maiores de idade, ou selecionar, para impressão, apenas o nome e a idade das pessoas. Ambas as operações são fundamentadas em conceitos da Álgebra Relacional:

A Seleção, consulta que filtra tuplas com base em um predicado booleano, define-se formalmente como:

$$\sigma_{\phi}(R) = \{ t \mid t \in R \wedge \phi(t) \} \quad (2)$$

Onde R é a relação original, t é uma tupla (linha da tabela original) que pertença a R e que satisfaça a condição imposta. A Projeção, operação que isola um subconjunto de atributos, define-se formalmente como:

$$\pi_L(R) = \{ t[L] \mid t \in R \} \quad (3)$$

Onde L são os atributos (colunas) que queremos selecionar da relação R original e $t[L]$ é a tupla resultante dos atributos selecionados (HEUSER, 2009). Codd define essas e outras operações relacionais, como projeção, união, interseção e diferença, como operações de conjuntos sobre os subconjuntos do produto cartesiano (CODD, 1970), evidenciando que o Modelo Relacional consolida a relação entre a Teoria dos Conjuntos e o domínio de dados no cenário computacional.

Além disso, na tentativa de padronizar o modelo relacional, Codd introduziu exigências que, para um leigo, possam parecer irrelevantes, mas que servem como pedra-mestra que confere um funcionamento regular ao banco de dados relacional (DATE, 2004). Observe abaixo as exigências e os problemas gerados pela negligência de tais decisões:

1. Compatibilidade de Esquemas na União: A exigência é de que a união de duas relações só é definida se elas possuírem o mesmo número de atributos com domínios correspondentes compatíveis, pois apenas assim o resultado constitui uma relação válida com esquema consistente (ELMASRI; NAVATHE, 2011). Essa restrição configura uma decisão de modelagem, e não uma propriedade da Teoria dos Conjuntos (DATE, 2004).

O exemplo a seguir ilustra o problema: ao tentar unir duas tabelas com esquemas incompatíveis (uma com coluna "Escola" e outra com coluna "Alergias"), o resultado seria uma relação mal formada com valores nulos em posições inconsistentes.

PK	Nome	Idade	Escola
1	Luis	15	A
2	Carlos	9	B
3	Amanda	12	C

Tabela 5a – Relação com atributo Escola

PK	Nome	Idade	Alergias
4	Antonio	22	Castanhas
5	Breno	14	Cachorro
6	Carlos	61	Medicamentos

Tabela 5b – Relação com atributo Alergias (esquema incompatível)

PK	Nome	Idade	Alergias	Escola
1	Luis	15		A
2	Carlos	9		B
3	Amanda	12		C
4	Antonio	22	Castanhas	
5	Breno	14	Cachorro	
6	Carlos	61	Medicamentos	

Tabela 5c – Resultado inválido da união de esquemas incompatíveis (linhas vermelhas: lacunas de domínio)

1. Restrição de Integridade de Entidade: Em um sistema relacional, diferentemente do estipulado na Teoria dos Conjuntos, não é possível introduzir $t_1 = \{1, Ana, 7\}$ a uma tabela cujo valor de chave primária 1 já mapeie uma instância distinta, visto que um único valor de chave primária mapearia duas instâncias distintas no mesmo corpo de dados (HEUSER, 2009).

Existem normalizações que previnem outros tipos de anomalias (HEUSER, 2009). Estas serão abordadas na seção seguinte. Stonebraker e Hellerstein (2005) documentam que a vantagem do Modelo Relacional sobre os demais se deveu à sua rigorosa base conjuntista. Sistemas que careciam de fundamentos matemáticos bem definidos não conseguiam garantir que a mesma consulta produzisse sempre o mesmo resultado, nem que operações equivalentes fossem reconhecidas como tais. Sob essa perspectiva, conclui-se que o conhecimento teórico, por mais críptico que seja, é fulcral para construir bases de dados confiáveis.

3.3 Formas Normais

O próximo passo para obter uma maestria sob o uso eficiente de dados é conhecer as Formas Normais, que são normalizações que garantem a ausência de anomalias de dados e otimizam a funcionalidade de um banco de dados. Antes de conceituar as 3 Formas Normais (note que, em algumas literaturas são expostas 5 ou mais Formas, mas as 3 primeiras já são muito boas para evitar a grande maioria de imbróglis), precisamos conceituar os tipos de dependência em um banco de dados e as categorias de atributos que causam problemas no nosso modelo.

A princípio, vamos discutir sobre as dependências encontradas em um banco de dados. Tomando como exemplo a tabela a seguir, é possível identificar 3 chaves primárias (ID_Aluno, ID_Materia e Semestre), uma vez que para acessar a nota, por exemplo, apenas o ID_Aluno não é suficiente para obter o resultado, ou seja, precisamos do valor de outros atributos para descobrir a informação.

ID_Aluno	ID_Matéria	Matéria	Semestre	Nota
222	1	Cálculo	1	8
222	2	Física	1	5
222	3	Álgebra	1	6
222	4	Química	1	10

Tabela 6 – Relação com Dependência Funcional Parcial: Nota depende de ID_Matéria e Semestre, não de ID_Aluno isoladamente

Este tipo de situação é conhecido como Dependência Funcional Parcial, quando meu atributo não-chave é identificável por meio de parte da minha chave primária, uma vez que as notas, nesta tabela, não dependem do ID_Aluno, mas sim do Semestre e ID_Materia.

Neste mesmo eixo, temos um caso de Dependência Funcional Transitiva quando um elemento é acessado por meio de um campo que não é a chave primária. Tendo como referência a tabela abaixo, vemos que a coluna "Funcao" não depende do ID_User (nossa chave primária), mas sim do "ID_Funcao"; logo, a coluna "Funcao" é redundante, uma vez que eu poderia removê-la e o banco de dados funcionaria da mesma forma, exigindo apenas a formalização de que o "ID_Funcao" 1 representa Aluno, e assim por diante.

ID_User	ID_Funcao	Funcao	Salário	Nome
1	1	Aluno	-1500	Auris
2	2	Professor	3000	Bento
3	1	Aluno	-1500	Cecilia
4	3	Diretor	7000	Alexandre

Tabela 7 – Relação com Dependência Funcional Transitiva: coluna "Funcao" depende de ID_Funcao, não de ID_User

O último conceito que é preciso dominar antes de compreender as Formas Normais são os Atributos Multivalorados e os Atributos Compostos. O registro abaixo conta com ambos os tipos de atributos:

CPF	Nome	Funcao	Telefone	Endereço
111	Marco	Aluno	(61) 99627-3615(61) 99372-2910	SQS 305, Bloco JBrasília, DF
222	Gael	Professor	(11) 98888-1111	Chácara 12, Conjunto BBrotas, SP
333	Davi	Aluno	(21) 97777-2222(21) 97777-3333(21) 97777-4444	Av. Atlântica, nº 1702Rio de Janeiro, RJ
444	Rafael	Professor	(31) 96666-5555	Rua da Bahia 10Belo Horizonte, MG

Tabela 8 – Atributos Multivalorados (Telefone) e Atributos Compostos (Endereço)

Neste caso, observamos a ocorrência de Atributos Multivalorados nos números de telefone, uma vez que os registros de PK 111 e 333 possuem mais de um valor no atributo telefone, configurando uma possível inconsistência no nosso modelo. Além disso, também ocorrem Atributos Compostos no atributo endereço, já que está sendo guardado a rua, a cidade e o estado do usuário, dados distintos no mesmo atributo.

Após discorrer acerca das anomalias e redundâncias frequentes em bancos de dados relacionais, faz-se imprescindível avaliar e conhecer as soluções para estes obstáculos para uma análise precisa de dados. As Formas Normais são processos compostos por procedimentos e boas práticas aplicadas para garantir que um banco de dados esteja bem estruturado. O objetivo central é evitar anomalias, originadas pela má estruturação da coleta e armazenamento de dados.

A Primeira Forma Normal (1FN) é específica para resolver problemas encontrados na tabela com atributos multivalorados. Deve-se seguir os seguintes passos:

1. Reconhecer o atributo multivalorado; no caso da tabela analisada, é a coluna "Telefone".
2. Criar uma nova tabela com uma chave (no caso, o CPF) e uma linha para cada atributo multivalorado.
3. Excluir da tabela original a coluna multivalorada.

Ao aplicar a 1FN, obtêm-se as seguintes tabelas:

CPF	Nome	Funcao	Endereço
111	Marco	Aluno	SQS 305, Bloco J Brasília, DF
222	Gael	Professor	Chácara 12, Conjunto B Brotas, SP
333	Davi	Aluno	Av. Atlântica, nº 1702 Rio de Janeiro, RJ
444	Rafael	Professor	Rua da Bahia 10 Belo Horizonte, MG

Tabela 9a – Tabela principal após aplicação da 1FN (coluna Telefone removida)

CPF	Telefone
111	(61) 99627-3615
111	(61) 99372-2910
222	(11) 98888-1111
333	(21) 97777-4444
333	(21) 97777-3333
333	(21) 97777-2222
444	(31) 96666-5555

Tabela 9b – Tabela de telefones criada pela 1FN, com uma linha por número

A Segunda Forma Normal (2FN) baseia-se na eliminação de Dependências Funcionais Parciais. Para que uma tabela esteja na 2FN, ela deve primeiramente satisfazer a 1FN e, adicionalmente, todos os atributos não-chave devem depender funcionalmente da chave primária completa, e não apenas de parte dela.

A Terceira Forma Normal (3FN) trata das Dependências Funcionais Transitivas. Para estar na 3FN, a tabela deve satisfazer a 2FN e, além disso, nenhum atributo não-chave pode depender de outro atributo não-chave. Atributos transitivamente dependentes devem ser extraídos para tabelas separadas, vinculadas por chaves estrangeiras.

3.4 Aplicação Prática: Corrigindo um Banco de Dados com Redundâncias por meio das Formas Normais

Após o desenvolvimento conceitual das Formas Normais, esta seção aplica os procedimentos a um banco de dados fictício composto por 1000 registros, simulando o cadastro de funcionários de uma empresa. A amostra a seguir, obtida por meio do comando `head(15)` em Python, ilustra a estrutura inicial dos dados:

#	CPF	Nome	ID_FUN	Funcao	Salario	Telefone	Endereço
0	46547	Gabriel Soares	3	Senior	7000	960981504	Av B, Bloco O - SP
1	19010	Leonardo Rocha	3	Senior	7000	904567934	Av B, Bloco K - PA
2	26430	Amanda Carvalho	1	Junior	2000	933801596, 953713502, 918048902	Av B, Bloco K - MG
3	23713	Pedro Lima	1	Junior	2000	929186941, 953900169	Alameda E, Bloco A - RJ

4	43857	Lucas Carvalho	2	Plenior	4500	932410532, 956966672, 910005473	Alameda E, Bloco K - DF
5	26379	Helena Silva	1	Junior	2000	932710976	Rua A, Bloco J - SP
6	38414	Amanda Vieira	0	Estagiario	1500	914838584	Alameda E, Bloco L - SP
7	29927	Larissa Martins	3	Senior	7000	973585221, 941687810	Alameda E, Bloco J - DF
8	16024	Gabriel Martins	2	Plenior	4500	925248179	Av B, Bloco L - MG
9	28049	Ricardo Souza	1	Junior	2000	943508258	Rua A, Bloco K - MG
10	13899	Beatriz Freitas	0	Estagiario	1500	917378745, 937118311	Rua A, Bloco O - MG
11	40744	Camila Silva	2	Plenior	4500	919233993, 919070673	Av B, Bloco L - PA
12	34228	Lucas Soares	2	Plenior	4500	937571090	Av B, Bloco J - PA
13	22160	Lucas Pereira	1	Junior	2000	966532080	Rua A, Bloco J - DF
14	20080	Rodrigo Santos	0	Estagiario	1500	938976484, 943588284	Alameda D, Bloco J - RJ

Tabela 10 – Amostra inicial do banco de dados fictício (head(15) sobre 1000 registros)

Note que o banco de dados original apresenta simultaneamente atributos compostos (coluna Endereço, que agrega rua, bloco e estado), atributos multivalorados (coluna Telefone, que armazena múltiplos números em uma mesma célula) e dependências funcionais transitivas (entre ID_FUNCAO, Funcao e Salario). Veja a seguir como polir o banco de dados fictício por meio da aplicação sucessiva das Formas Normais, fundamentadas nos conceitos abordados na seção 3.2 (Álgebra Relacional) do projeto.

3.4.1 Aplicação da Primeira Forma Normal: Atributos Compostos

A fim de remover as inconsistências e anomalias estruturais, deve-se começar empregando a Primeira Forma Normal (1FN). Utilizando o comando head(), são apresentadas as primeiras cinco linhas do banco de dados, nas quais é possível identificar a existência de atributos compostos na coluna Endereço, que contém informações de avenida/rua, bloco e estado agrupadas em um único campo. A 1FN corrige esse imbróglio da seguinte forma: cada informação deve ser salva em um atributo específico. No caso apresentado, serão adicionadas ao banco de dados três colunas: Rua, Bloco e Estado, e, simultaneamente, será removida a coluna Endereço.

CPF	Nome	ID_F UN	Funcao	Salario	Telefone	Endereço
46547	Gabriel Soares	3	Senior	7000	960981504	Av B, Bloco O - SP
19010	Leonardo Rocha	3	Senior	7000	904567934	Av B, Bloco K - PA
26430	Amanda Carvalho	1	Junior	2000	933801596	Av B, Bloco K - MG
23713	Pedro Lima	1	Junior	2000	929186941	Alameda E, Bloco A - RJ
43857	Lucas Carvalho	2	Plenior	4500	932410532	Alameda E, Bloco K - DF

Tabela 11 – Banco de dados antes da 1FN: coluna Endereço com atributos compostos (rua, bloco e estado em uma única célula). *Amostra obtida por meio do head() em Python.

Após a decomposição do atributo composto, obtém-se o seguinte resultado:

CPF	Nome	ID_FU N	Funcao	Salario	Telefone	Rua	Bloco	Estado
46547	Gabriel Soares	3	Senior	7000	960981504	Av B	O	SP
19010	Leonardo Rocha	3	Senior	7000	904567934	Av B	K	PA
26430	Amanda Carvalho	1	Junior	2000	933801596	Av B	K	MG
23713	Pedro Lima	1	Junior	2000	929186941	Alameda E	A	RJ
43857	Lucas Carvalho	2	Plenior	4500	932410532	Alameda E	K	DF

Tabela 12 – Banco de dados após a primeira aplicação da 1FN: atributo Endereço decomposto em Rua, Bloco e Estado. *Amostra obtida por meio do head() em Python.

3.4.2 Aplicação da Primeira Forma Normal: Atributos Multivalorados

A segunda anomalia corrigida pela Primeira Forma Normal é a existência de atributos multivalorados, ou seja, mais de um dado do mesmo tipo armazenado em uma única coluna. Trata-se de uma redundância que compromete a integridade dos dados, e a correção segue os seguintes passos:

1. Reconhecer qual coluna possui atributos multivalorados.
2. Criar uma nova tabela que guarde, em cada tupla, a chave primária da tabela original e um único valor do atributo multivalorado.
3. Remover o atributo multivalorado da tabela original.

Observe o esquema a seguir, no qual o atributo Telefone contém mais de um registro em alguns casos:

CPF	Nome	ID_FU N	Funcao	Salario	Telefone	Rua	Bloco	Estado
29927	Larissa Martins	3	Senior	7000	973585221, 941687810	Alameda E	J	DF
49814	Matheus Dias	3	Senior	7000	972048923, 931812922	Rua A	A	MG
46690	Bruno Freitas	2	Plenior	4500	924787189, 985823728	Alameda E	O	PA
36371	Gabriel Andrade	3	Senior	7000	927290608, 930628429	Alameda E	L	MG
37641	Bárbara Santos	1	Junior	2000	935097075, 939569528	Alameda C	L	MG

Tabela 13 – Banco de dados antes da segunda aplicação da 1FN: atributo Telefone multivalorado.

Após a aplicação da 1FN para atributos multivalorados, obtêm-se as seguintes tabelas:

CPF	Nome	ID_FU N	Funcao	Salario	Rua	Bloco	Estado
29927	Larissa Martins	3	Senior	7000	Alameda E	J	DF
49814	Matheus Dias	3	Senior	7000	Rua A	A	MG
46690	Bruno Freitas	2	Plenior	4500	Alameda E	O	PA
36371	Gabriel Andrade	3	Senior	7000	Alameda E	L	MG
37641	Bárbara Santos	1	Junior	2000	Alameda C	L	MG

Tabela 14a – Tabela principal após remoção do atributo Telefone.

CPF (FK)	Telefone
29927	973585221
29927	941687810
49814	972048923
49814	931812922
46690	924787189
46690	985823728
36371	927290608
36371	930628429
37641	935097075
37641	939569528

Tabela 14b – Nova tabela Telefone, criada pela 1FN, com uma linha por número e CPF atuando como chave estrangeira.

3.4.3 Aplicação da Segunda Forma Normal: Eliminação de Dependências Parciais

A Segunda Forma Normal (2FN) é aplicada em bancos com mais de uma chave primária e garante que um atributo não chave depende inteiramente da chave primária, não de parte dela.

CPF (PK)	ID_FUNC (PK)	Nome	Funcao	Salario	ID_PROJ (PK)	NOME PROJ
29927	3	Larissa Martins	Senior	7000	1	Estudo de Redes
29927	3	Larissa Martins	Senior	7000	2	Diagramas de Venn
49814	3	Matheus Dias	Senior	7000	3	Uso de IA
49814	3	Matheus Dias	Senior	7000	2	Diagramas de Venn
46690	2	Bruno Freitas	Plenior	4500	4	Plano Negociação

Tabela 15 – Cenário hipotético com violação da 2FN: Nome, Funcao e Salario dependem apenas de CPF, não da chave composta (CPF, Telefone).

Observe que o atributo nome não depende do ID_FUNC, apenas do CPF, o atributo Nome_PROJ depende apenas do ID_PROJ e o atributo salário depende apenas do ID_FUNC, logo, trata-se, de uma Dependência Funcional Parcial: cada vez que Larissa Martins inicia um novo projeto, seu nome, função e salário são repetidos desnecessariamente, gerando redundância, o que ocupa mais espaço do armazenamento. A correção pela 2FN consiste em separar a relação em duas tabelas: uma contendo apenas os atributos que dependem da chave completa, e outra contendo os atributos que dependem somente da parte CPF.

CPF (PK,FK)	ID_PROJ
29927	1
29927	2
49814	3
49814	3
46690	4

Tabela 16a – Tabela Telefone após a 2FN: apenas a chave composta, sem atributos parcialmente dependentes.

CPF (PK)	Nome	Funcao	Salario
29927	Larissa Martins	Senior	7000
49814	Matheus Dias	Senior	7000
46690	Bruno Freitas	Plenior	4500

Tabela 16b – Tabela Pessoa após a 2FN: atributos que dependem integralmente do CPF.

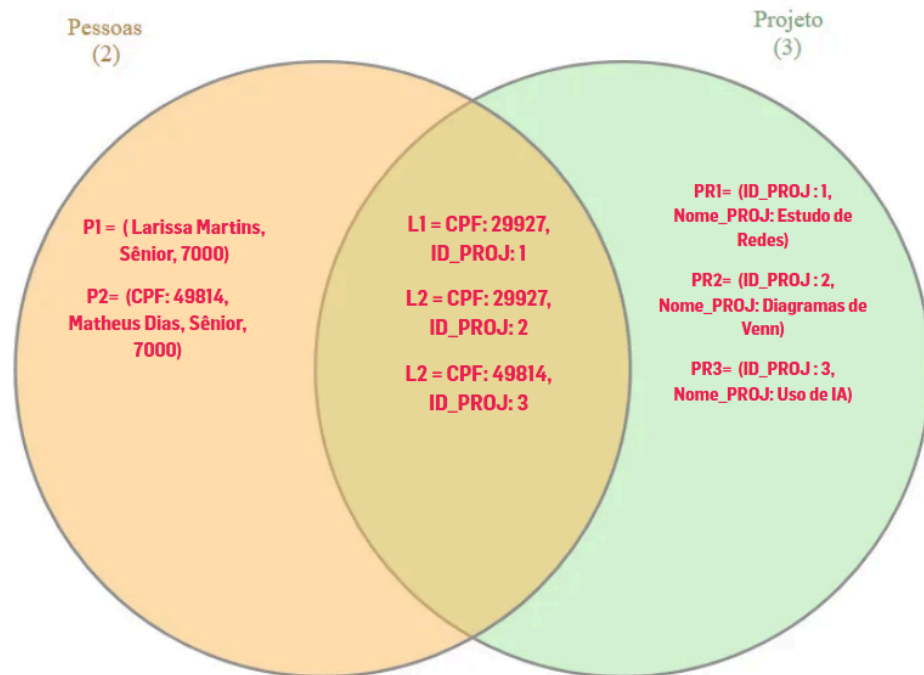
ID_PROJ(PK, FK)	NomePROJ
1	Estudo de Redes
2	Diagramas de Venn
3	Uso de IA
4	Plano de Negociação

Tabela 16c – Tabela Pessoa após a 2FN: atributos que dependem integralmente do ID_PROJ.

É importante destacar que, na implementação direta da 1FN apresentada nas seções 3.4.1 e 3.4.2, a tabela Telefone foi criada já em conformidade com a 2FN (contendo apenas a chave composta, sem atributos não-chave), de modo que o banco de dados decorrente daquela

aplicação já satisfaz automaticamente a Segunda Forma Normal. O cenário hipotético acima foi utilizado apenas para ilustrar didaticamente a natureza da violação.

Abaixo, a representação da correção visualizada por meio de um diagrama de Venn:



*Diagrama feito no site www.interactivenn.net e canva.com

Diagrama 1 – Diagrama da correção realizada pela segunda forma normal

Atente-se ao fato de que:

1. A tabela possui menos registros para simplificar a visualização
2. A interseção entre os 2 conjuntos (tabelas) seria uma tabela referencial que faz a ligação entre um CPF e o ID_PROJ

3.4.4 Aplicação da Terceira Forma Normal: Eliminação de Dependências Transitivas

A Terceira Forma Normal (3FN) ataca uma redundância sutil, porém recorrente: a Dependência Funcional Transitiva, em que um atributo não-chave depende de outro atributo não-chave, e não diretamente da chave primária. No banco de dados em análise, observe a seguinte amostra extraída da tabela principal já em 1FN:

CPF	Nome	ID_FUNC AO	Funcao	Salario	Rua	Bloco	Estado
46547	Gabriel Soares	3	Senior	7000	Av B	O	SP
19010	Leonardo Rocha	3	Senior	7000	Av B	K	PA
26430	Amanda Carvalho	1	Junior	2000	Av B	K	MG
23713	Pedro Lima	1	Junior	2000	Alameda E	A	RJ
43857	Lucas Carvalho	2	Plenior	4500	Alameda E	K	DF

Tabela 17 – Tabela principal em 1FN com violação da 3FN: Dependência transitiva

Verifica-se as seguintes dependências: o CPF identifica unicamente cada funcionário (PK) e determina seu ID_FUNCAO; o ID_FUNCAO determina a Função (ID_FUNCAO 3 sempre corresponde a Senior, ID_FUNCAO 1 sempre corresponde a Junior, etc); e a Função determina o Salario (Senior Junior, Plenior e Estagiario recebem salários fixos). Portanto, Funcao e Salario são dependentes do CPF por meio do ID_FUNCAO.

As consequências práticas dessa redundância ocorrem, precipuamente, na hora de alterar algum valor do banco de dados. Por exemplo, caso o salário de um cargo seja reajustado, cada linha que represente um funcionário daquele cargo precisará ser atualizada individualmente, consumindo tempo, recursos e aumentando a margem para erros. A correção pela 3FN consiste em extrair os atributos transitivamente dependentes para uma tabela separada de lookup, mantendo na tabela principal apenas a chave estrangeira:

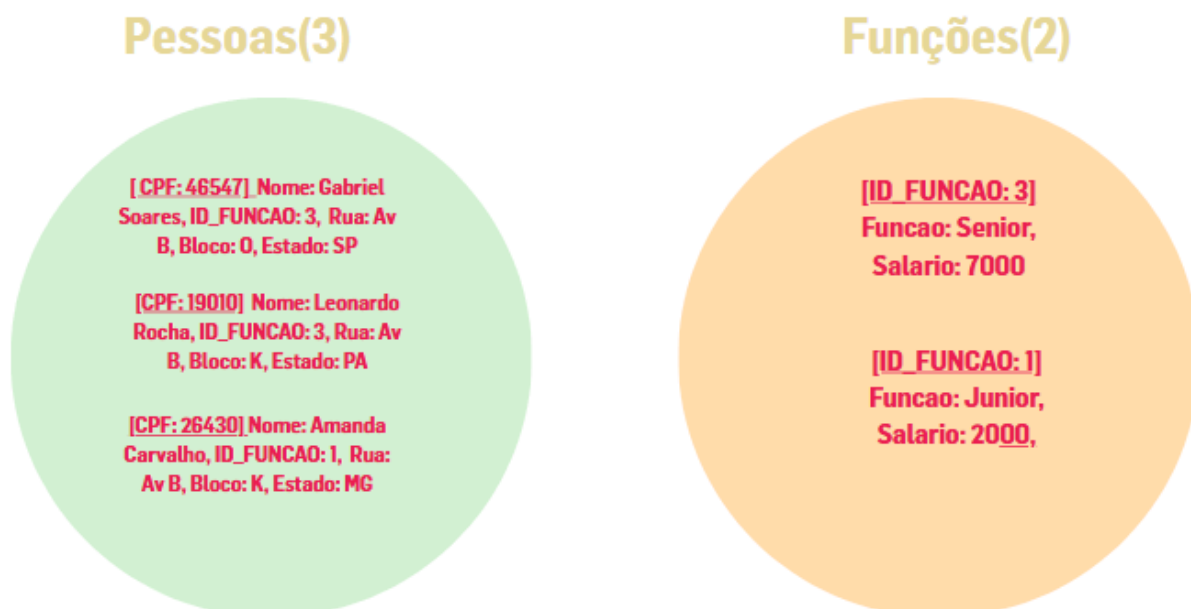
CPF (PK)	Nome	ID_FUNC O (FK)	Rua	Bloco	Estado
46547	Gabriel Soares	3	Av B	O	SP
19010	Leonardo Rocha	3	Av B	K	PA
26430	Amanda Carvalho	1	Av B	K	MG
23713	Pedro Lima	1	Alameda E	A	RJ
43857	Lucas Carvalho	2	Alameda E	K	DF
26379	Helena Silva	1	Rua A	J	SP

Tabela 18a – Tabela Pessoa após a 3FN: Funcao e Salario foram removidos, restando apenas a chave estrangeira ID_FUNCAO.

ID_FUNCAO (PK)	Funcao	Salario
0	Estagiario	1500
1	Junior	2000
2	Plenior	4500
3	Senior	7000
4	CEO	10000

Tabela 18b – Tabela Funcao após a 3FN: catálogo único contendo a descrição da função e seu salário associado.

A seguir, a correção representada por um diagrama de Venn. Note que não existe interseção entre os conjuntos, apenas a disjunção do conjunto principal, que era a tabela que não seguia a 3FN, em 2 subconjuntos de Pessoas, cuja Chave Primária (PK) é o CPF e a Chave Estrangeira (FK) é o ID_FUNÇÃO e o subconjunto Funções, que por meio da FK_ID_FUNCAO, faz referência à tabela de pessoas, garantindo integridade e a eliminação de redundâncias.



*Diagrama feito no site [canva.com](https://www.canva.com)

Diagrama 2 – Disjunção da tabela, ilustrando a correção feita pela terceira forma normal

3.4.5 Banco de Dados Totalmente Normalizado

Após a aplicação sucessiva da 1FN (para atributos compostos e multivalorados), seguida pela 2FN (verificação de dependências parciais) e 3FN (eliminação da cadeia transitiva $ID_FUNCAO \rightarrow Funcao \rightarrow Salario$), o banco de dados resulta em três relações independentes e logicamente íntegras. A seguir, apresenta-se uma amostra obtida pelo comando `head(6)` sobre cada uma das tabelas resultantes:

CPF (PK)	Nome	ID_FUNCAO (FK)	Rua	Bloco	Estado
46547	Gabriel Soares	3	Av B	O	SP
19010	Leonardo Rocha	3	Av B	K	PA
26430	Amanda Carvalho	1	Av B	K	MG
23713	Pedro Lima	1	Alameda E	A	RJ
43857	Lucas Carvalho	2	Alameda E	K	DF
26379	Helena Silva	1	Rua A	J	SP

Tabela 19a – Tabela Pessoa (`head(6)`): atributos diretamente dependentes do CPF.

CPF (FK)	Telefone (PK)
26430	933801596
26430	953713502
26430	918048902
23713	929186941
23713	953900169
43857	932410532

Tabela 19b – Tabela Telefone (`head(6)`): vínculo entre CPF e cada número individual.

ID_FUNCAO (PK)	Funcao	Salario
0	Estagiario	1500
1	Junior	2000
2	Plenior	4500
3	Senior	7000
4	CEO	10000

Tabela 19c – Tabela Funcao: catálogo de cargos e salários, referenciado pela chave estrangeira ID_FUNCAO.

Ao fim, sobraram no total 3 tabelas que respeitam totalmente as lógicas propostas pela normatizações e pela teoria da Álgebra Relacional, fundamentada na teoria dos conjuntos. Note que, com o auxílio da representação da correção por meio do uso de uma tabela exemplo facilita a compreensão de sua aplicação, entretanto, o entendimento teórico sem o uso desse instrumento pode se tornar muito abstrato. Portanto, a inclusão de Diagramas de Venn na análise da validade lógica da estrutura de um banco de dados relacional é imprescindível.

Entretanto, para realizar essa aplicação com êxito, é necessário reconhecer suas limitações. De acordo com o que foi exposto, a 1ºFN, por não se tratar de um problema causado por relações de pertencimento, mas sim por agregação de valores, não pode ser resolvida ou ilustrada por meio de um diagrama de Venn convencional. Além disso, algumas correções, como a realizada pela 3ºFN é limitada quanto ao uso do diagrama, visto que não existe uma relação geométrica de interseção, apenas de disjunção da tabela/conjunto original.

3.5 Validação Lógica de Operações SQL por meio de Diagramas de Venn

O MySQL é um Sistema de Gerenciamento de Banco de Dados Relacional (SGBDR), ferramenta utilizada para realizar consultas, incluir e modificar dados estruturados. Os SGBDs são essenciais na computação moderna por permitirem níveis granulares de acesso e privilégios aos dados, ou seja, restrições acerca de quem pode ou não ver e editar certos dados, garantindo um controle seguro e eficiente sobre grandes volumes de informações. Atualmente, o MySQL destaca-se como um dos SGBDs mais difundidos no mercado devido à sua natureza de código open-source, gratuidade e suporte a diversas linguagens de programação. Além disso, o MySQL trabalha com estruturas de dados relacionais, garantindo maior integridade, a sigla ACID (Atomicidade, Consistência, Isolamento, Durabilidade) descreve os pilares dessa estrutura, e facilidade na execução de consultas.

No que tange à temática deste artigo, é válido ilustrar a forma com que o MySQL executa suas funções de manipulação de dados para que, conseqüentemente, seja visível a participação ativa dos Diagramas de Venn como ferramentas auxiliares na validação lógica dos comandos. A análise a seguir demonstrará como os comandos lógicos do software são reflexos diretos dos operadores da Álgebra Relacional e da Teoria dos Conjuntos, tornando explícita a viabilidade dos Diagramas de Venn como instrumentos de validação dessas operações. Para fins de exemplificação, utilizar-se-á a Tabela 19a (Pessoa) como referência.

3.5.1 WHERE: Implementação da Operação de Seleção

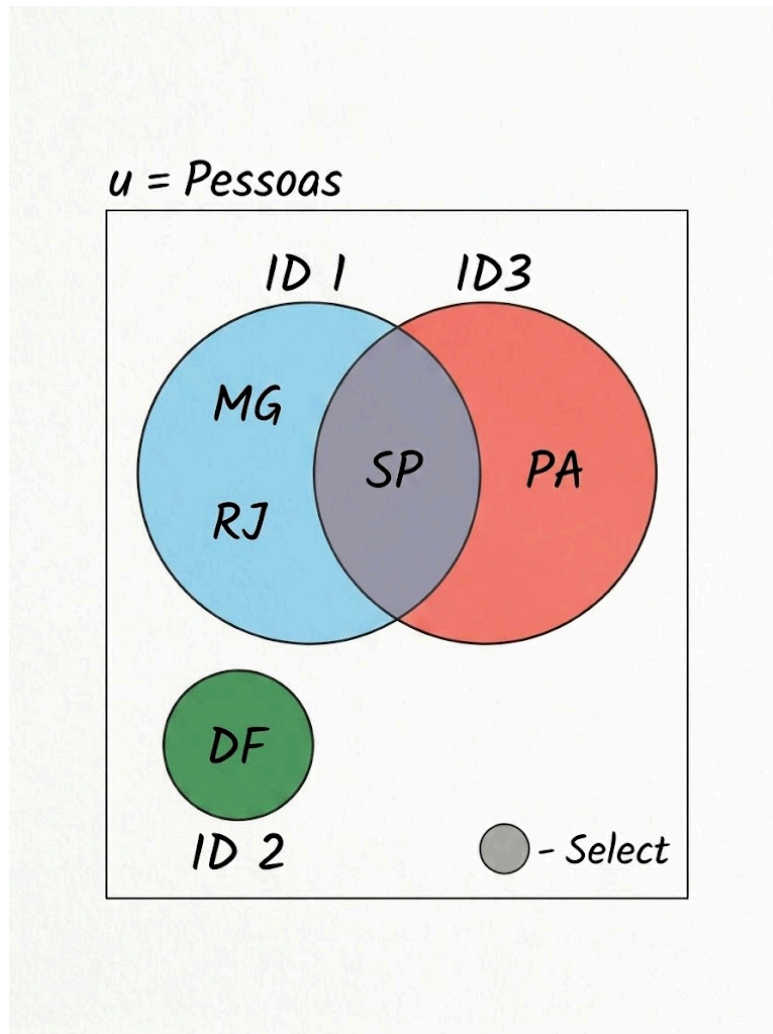
No MySQL, o comando WHERE implementa a operação de seleção, sendo este um pilar essencial para filtrar as tuplas de uma tabela de modo que apenas registros que atendam a uma condição específica sejam exibidos. Tomando como referência a Tabela 19a (Pessoa), caso o desenvolvedor queira obter os registros sediados no estado de São Paulo, pode-se executar o seguinte comando:

```
SELECT * FROM Pessoa WHERE Estado = 'SP';
```

Esse comando exibirá apenas as tuplas cujo atributo Estado esteja preenchido com 'SP'. Em termos da Teoria dos Conjuntos, a operação corresponde a:

$$R' = \{ t \in \text{Pessoa} \mid t.\text{Estado} = \text{'SP'} \} \quad (4)$$

O uso de Diagramas de Venn é uma ferramenta que facilita a visualização desse tipo de consulta, pois permite que o usuário ou programador observe, de forma palpável, o subconjunto resultante da operação de seleção descrita pela equação (4).



**Diagrama desenhado à mão e digitalizado pelo gemini, com base na tabela 19a*

Diagrama 3a – Ilustração da operação SELECT

3.5.2 JOIN: Implementação das Operações de Junção e Combinação

O comando JOIN no MySQL implementa as operações de junção da Álgebra Relacional, combinando tuplas de duas ou mais relações com base em atributos comuns. Diferentemente das operações de seleção e projeção, que atuam internamente sobre uma única relação, o JOIN realiza a combinação de domínios distintos, sendo a operação que mais se beneficia da validação visual por meio de Diagramas de Venn, conforme demonstrado por Atwood (2007). Existem quatro variantes principais, cada uma correspondendo a uma região específica do diagrama formado pelos dois conjuntos envolvidos.

INNER JOIN: corresponde à interseção ($A \cap B$) entre duas relações, retornando apenas as tuplas cuja correspondência exista em ambas as tabelas. Tomando como base as

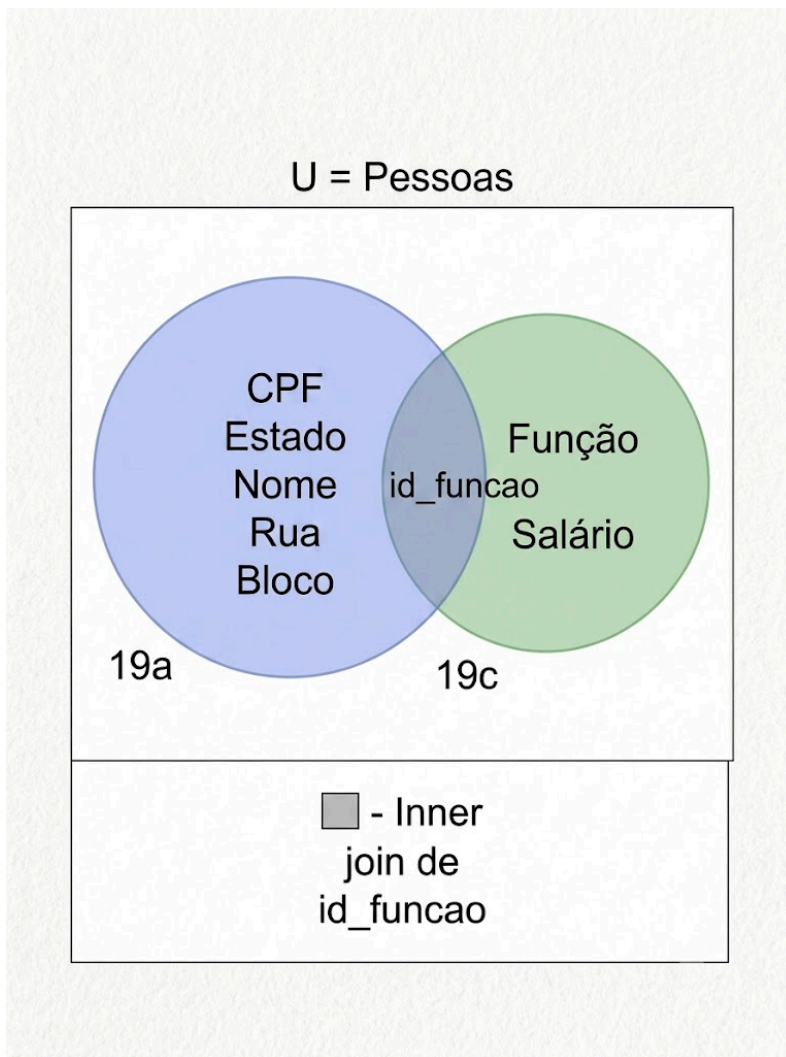
Tabelas 19a (Pessoa) e 19c (Funcao), o comando a seguir retorna todos os funcionários acompanhados das descrições de seus respectivos cargos:

```
SELECT P.Nome, F.Funcao, F.Salario  
FROM Pessoa P INNER JOIN Funcao F ON P.ID_FUNCAO = F.ID_FUNCAO;
```

Em notação de Álgebra Relacional, a operação corresponde a:

$$R \bowtie_{\theta} S = \sigma_{\theta}(R \times S) \quad (5)$$

Onde θ é a condição de junção ($P.ID_FUNCAO = F.ID_FUNCAO$). O resultado da operação é, portanto, uma nova relação que contém os conteúdos de ambas as tabelas, um subconjunto do produto cartesiano filtrado pela igualdade dos atributos de ligação.



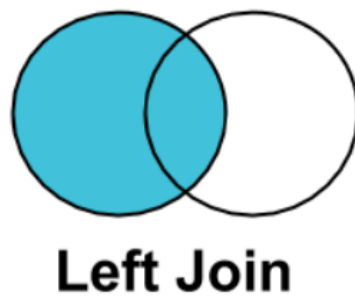
**Diagrama desenhado à mão e digitalizado pelo gemini, com base na tabela 19a e 19c*

Diagrama 3b – Ilustração da operação INNER JOIN

LEFT JOIN (ou LEFT OUTER JOIN): retorna todas as tuplas da relação à esquerda da operação, mesmo aquelas que não possuam correspondência na relação à direita. Os atributos da relação à direita são preenchidos com NULL quando a correspondência for inexistente. Formalmente, representa a união da interseção com a região exclusiva de A, isto é, $(A \cap B) \cup (A \setminus B)$, o que equivale ao próprio conjunto A com extensões nulas:

$$R \bowtie_{\theta} S \quad (6)$$

Esse tipo de junção é útil, por exemplo, para listar todos os funcionários da tabela Pessoa juntamente com seus telefones, caso existam, sem perder os funcionários que não possuem telefone cadastrado. De maneira mais simples, o LEFT JOIN vai integrar à relação da esquerda.



*Fonte: <https://docs.agilebase.co.uk/docs/views/joins/types-of-join/>

Diagrama 3c – Ilustração da operação LEFT JOIN

Após o Left Join, a tabela obtida a partir das tabelas 19a e 19c conterá valores nulos no atributo TELEFONE para usuários sem telefone cadastrado. Observe o exemplo abaixo:

CPF	Nome	ID_FUNCAO	Rua	Bloco	Estado	Telefone
46547	Gabriel Soares	3	Av B	O	SP	NULL
19010	Leonardo Rocha	3	Av B	K	PA	NULL
26430	Amanda Carvalh	1	Av B	K	MG	933801596
26430	Amanda Carvalh	1	Av B	K	MG	953713502
26430	Amanda Carvalh	1	Av B	K	MG	918048902

Tabela 20a – Tabela obtida após o LEFT JOIN da tabela 19c e 19a. Mostrando os atributos NULOS quando não existem registros daquele tipo

RIGHT JOIN (ou RIGHT OUTER JOIN): é a operação espelhada do LEFT JOIN: retorna todas as tuplas da relação à direita, mesmo as que não tenham correspondência na relação à esquerda, preenchendo com NULL os atributos faltantes.

$$R \bowtie_{\theta} S \quad (7)$$

FULL OUTER JOIN: retorna todas as tuplas de ambas as relações, com preenchimento de NULL onde não houver correspondência. É união completa dos dois conjuntos ($A \cup B$), incluindo as três regiões: $A \setminus B$, $A \cap B$ e $B \setminus A$:

$$R \bowtie_{\theta} S \quad (8)$$

Esta é a operação que mais explicitamente visualiza o uso dos Diagramas de Venn como ferramenta de verificação, uma vez que abarca todas as regiões do diagrama de dois conjuntos descritas formalmente na Seção 4.3 deste trabalho. Para ilustrar o resultado de um FULL OUTER JOIN, vamos utilizar a tabela 15 e a 19a. Como o FULL OUTER JOIN devolve a união completa (3 regiões) de todas as tabelas envolvidas. O resultado, seria:

CPF	ID_FUNCAO	Nome	Função	Salário	ID_PROJ	Nome_Proj	Rua	Bloco	Estado
29927	3	Larissa Martins	Senior	7000	1	Estudo de Rede:	NULL	NULL	NULL
29927	3	Larissa Martins	Senior	7000	2	Diagramas de Ve	NULL	NULL	NULL
49814	3	Matheus Dias	Senior	7000	3	Uso de IA	NULL	NULL	NULL
49814	3	Matheus Dias	Senior	7000	2	Diagramas de Ve	NULL	NULL	NULL
46690	2	Bruno Freitas	Plenior	4500	4	Plano Negociação	NULL	NULL	NULL
46547	3	Gabriel Soares	NULL	NULL	NULL	NULL	Av B	O	SP
19010	3	Leonardo Rocha	NULL	NULL	NULL	NULL	Av B	K	PA
26430	1	Amanda Carvalh	NULL	NULL	NULL	NULL	Av B	K	MG
23713	1	Pedro Lima	NULL	NULL	NULL	NULL	Alameda E	A	RJ
43857	2	Lucas Carvalho	NULL	NULL	NULL	NULL	Alameda E	K	DF
26379	1	Helena Silva	NULL	NULL	NULL	NULL	Rua A	J	SP

Tabela 20b – Tabela obtida após o FULL OUTER JOIN da tabela 15 e 19a. Mostrando os atributos NULLS quando não existem registros daquele tipo

3.5.3 SELECT: Implementação da Operação de Projeção

Por fim, o comando SELECT implementa a operação de projeção: trata-se de uma operação que filtra a tabela verticalmente, selecionando apenas os atributos desejados. Observe que, na Tabela 19a (Pessoa), o comando a seguir:

SELECT CPF, Nome FROM Pessoa;

pode ser reescrito, de forma críptica para alguém que não domine plenamente a Teoria dos Conjuntos, como:

$$R' = \{ (t.CPF, t.Nome) \mid t \in \text{Pessoa} \} \quad (9)$$

Atente-se, contudo, que dentre todas as consultas exemplificadas neste bloco, o SELECT é a única que não tem uma correspondência direta em um Diagrama de Venn, uma vez que altera a dimensão (ou seja, o número de atributos) da tabela exibida, tornando-se incompatível com o propósito do diagrama, cujo objetivo é representar a relação de pertencimento a um conjunto, conceito melhor explicado na Seção 2.2 deste trabalho.

4 CONCLUSÃO

Foi observado, portanto, que o uso de diagramas de Venn é restrito para algumas consultas SQL e para a validação de algumas Formas Normais. Para a análise de interações que extrapolam o escopo de pertencimento, a ferramenta é inútil, mas, quando estamos tratando de situações de pertencimento, o uso dos diagramas de Venn facilitam a visualização e modelagem lógica do banco de dados. Logo, que o uso dos diagramas seja proveitoso para um projeto de banco de dados relacional, é imprescindível reconhecer as dimensões tanto do projeto quanto do ferramental, uma vez que, por exemplo, a 1FN, por não se tratar de um problema causado por relações de pertencimento, não pode ser modelada com um diagrama de Venn. Essa limitação prova que, ferramentas como MERs (Modelo de Entidade Relacional) tradicionais, como o oferecido pelo MySQL, tanto em seu modelo lógico como físico, são mais proveitosos e recomendados para a construção e visualização da lógica por trás de entidades relacionais, entretanto, o uso de diagramas de Venn ainda pode ser útil para validar e explicar consultas, como WHERE e JOIN, e correções das Formas Normais, sobretudo, da 2FN.

5 DEFINIÇÕES FORMAIS CENTRAIS

5.1 Símbolos e Operações de Conjuntos

Símbolos de conjuntos numéricos:

- $\mathbb{N}$: Conjunto dos números Naturais $\{0, 1, 2, 3, \dots\}$.
- $\mathbb{Z}$: Conjunto dos números Inteiros $\{\dots, -2, -1, 0, 1, 2, \dots\}$.
- $\mathbb{Q}$: Conjunto dos números Racionais (definidos por frações a/b).
- $\mathbb{I}$ ou $\mathbb{R} \setminus \mathbb{Q}$: Conjunto dos números Irracionais (decimais infinitos não periódicos).
- $\mathbb{R}$: Conjunto dos números Reais (união dos racionais com irracionais).
- $\mathbb{C}$: Conjunto dos números Complexos.
- U : Conjunto Universo.

Operações com conjuntos:

- União ($\cup$): $A \cup B = \{x \mid x \in A \vee x \in B\}$
- Interseção ($\cap$): $A \cap B = \{x \mid x \in A \wedge x \in B\}$
- Diferença ($-$ ou $\setminus$): $A - B = \{x \mid x \in A \wedge x \notin B\}$
- Complementar (A^c): $\{x \in U \mid x \notin A\}$

Propriedades:

- $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$
- $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$
- $(A \cup B)^c = A^c \cap B^c$ [Lei de De Morgan]
- $(A \cap B)^c = A^c \cup B^c$ [Lei de De Morgan]

Relação de conjuntos (inclusão):

- Está contido ($\subset$ ou $\subseteq$): $A \subset B \Leftrightarrow \forall x(x \in A \Rightarrow x \in B)$
- Contém ($\supset$ ou $\supseteq$): $B \supset A$ = indica que B possui todos os elementos de A.
- Não está contido ($\not\subset$): $\exists x(x \in A \wedge x \notin B)$
- Conjunto Vazio ($\emptyset$): $\{x \mid x \neq x\}$

5.2 Fundamentação Axiomática

Axioma da Extensionalidade: $A = B \Leftrightarrow \forall x(x \in A \leftrightarrow x \in B)$. Define que dois conjuntos são iguais se, e somente se, possuem os mesmos elementos. É a base para as relações de igualdade e inclusão.

Axioma do Par / União: $\forall A \forall B \exists C \forall x(x \in C \leftrightarrow (x \in A \vee x \in B))$. Garante que, dados dois conjuntos, a união deles também resulta em um conjunto válido dentro do sistema matemático.

Produto Cartesiano: Dados dois conjuntos A e B, o produto cartesiano de A e B é o conjunto $A \times B = \{(a, b) \mid a \in A \wedge b \in B\}$. O produto cartesiano é o conjunto de todos os pares ordenados (a, b) com a em A e b em B. Esta é a definição que Codd (1970, seção 1.1) transpõe diretamente para definir relações de banco de dados.

5.3 Regiões do Diagrama de Venn

Para dois conjuntos A e B, definem-se formalmente as quatro regiões:

- $R1 = A \setminus B = \{x \mid x \in A \wedge x \notin B\}$
- $R2 = A \cap B = \{x \mid x \in A \wedge x \in B\}$
- $R3 = B \setminus A = \{x \mid x \notin A \wedge x \in B\}$
- $R4 = (A \cup B)^c = \{x \in U \mid x \notin A \wedge x \notin B\}$

5.4 Estruturas e Operações da Álgebra Relacional

Estruturas relacionais:

- Relação: subconjunto do produto cartesiano entre diferentes domínios $D1, D2, \dots$; formalmente $R \subseteq D1 \times D2 \times \dots$
- Tupla: elemento pertencente a uma relação R ($t \in R$). Em bancos de dados relacionais, são representadas pelas linhas.
- Atributo: descrição dos dados armazenados em uma relação, associado a um domínio específico que restringe o tipo de valores armazenados em cada coluna.
- Domínio: conjunto de valores válidos que podem ser armazenados em um atributo.

Operações relacionais:

- Seleção: $\sigma\phi(R) = \{ t \mid t \in R \wedge \phi(t) \}$ = filtra tuplas com base em um predicado booleano.
- Projeção: $\pi L(R) = \{ t[L] \mid t \in R \}$ = isola um subconjunto de atributos.
- Produto Cartesiano: $R \times S = \{(r,s) \mid r \in R \wedge s \in S\}$
- Junção (Join): $\sigma\phi(R \times S) = (R \bowtie_{\theta} S)$, resultado do produto cartesiano com filtro aplicado.

5.5 Estruturas do Modelo Relacional e SQL

- Tabela: representa logicamente uma relação R. A ordem das linhas é irrelevante e, em tese, os elementos não são duplicados.
- Colunas: representam os atributos de uma tabela, possuindo um domínio definido que delimita os valores válidos.
- Chave Primária (PK): atributo que dá, a uma tupla, uma identificação única. Formalmente: $t_1, t_2 \in R$, se $t_1 \neq t_2$, então $t_1[PK] \neq t_2[PK]$.
- Chave Estrangeira (FK): atributo que referencia a PK em uma segunda relação, garantindo a integridade referencial. Para que FK exista em R2, ele necessariamente deve existir em R1.
- SELECT: implementa a operação de projeção (π).
- WHERE: implementa a operação de seleção (σ).
- UNION: realiza a união de conjuntos ($\cup$) entre duas relações com domínios compatíveis.
- JOIN (INNER): retorna $R \bowtie_{\theta} S$.
- JOIN (OUTER LEFT): retorna todas as tuplas de R; se não houver correspondência em S, os atributos recebem NULL.
- JOIN (OUTER RIGHT): retorna todas as tuplas de S; se não houver correspondência em R, os atributos recebem NULL.
- JOIN (OUTER FULL): retorna todas as tuplas de R e S, aplicando NULL onde a correspondência for inexistente.

REFERÊNCIAS

- ATWOOD, Jeff. A Visual Explanation of SQL Joins. Coding Horror, 2007. Disponível em: <https://blog.codinghorror.com/a-visual-explanation-of-sql-joins/>. Acesso em: 16 maio 2026.
- BAGARIA, Joan. Set Theory. In: ZALTA, Edward N. (ed.). The Stanford Encyclopedia of Philosophy. Stanford: Stanford University, 2023. Disponível em: <https://plato.stanford.edu/entries/set-theory/>. Acesso em: 16 maio 2026.
- BEAULIEU, Alan. Learning SQL: Generate, Manipulate, and Retrieve Data. 3. ed. Sebastopol: O'Reilly Media, 2020.
- CHAMBERLIN, Donald D. Early History of SQL. IEEE Annals of the History of Computing, v. 34, n. 4, p. 78-82, out./dez. 2012.
- CODD, E. F. A Relational Model of Data for Large Shared Data Banks. Communications of the ACM, v. 13, n. 6, p. 377–387, 1970.
- DATE, C. J. Introdução a Sistemas de Bancos de Dados. Rio de Janeiro: Elsevier, 2004.
- DATE, C. J. SQL e Teoria Relacional: como escrever código SQL preciso. Tradução de Edson Furmankiewicz. São Paulo: Novatec, 2012.
- FERREIRA, Jaime Campos. Elementos de lógica matemática e teoria dos conjuntos. Lisboa: Departamento de Matemática do Instituto Superior Técnico, 2001. Disponível em: <https://www.ime.usp.br/mat/206/textos/elmtcJaimeCamposFerreira.pdf>. Acesso em: 18 maio 2026.
- HEUSER, Carlos Alberto. Projeto de Banco de Dados. 6. ed. Porto Alegre: Bookman, 2009.
- IBM. O que é a redundância de dados? [S. l.]: IBM Think, 2024. Disponível em: <https://www.ibm.com/br-pt/think/topics/data-redundancy>. Acesso em: 18 maio 2026.
- IBM. O que é normalização de banco de dados? [S. l.]: IBM Think, 2024. Disponível em: <https://www.ibm.com/br-pt/think/topics/database-normalization>. Acesso em: 18 maio 2026.
- KENT, William. A Simple Guide to Five Normal Forms in Relational Database Theory. Communications of the ACM, New York, v. 26, n. 2, p. 120-125, fev. 1983.

LOUREIRO, Antonio Alfredo Ferreira. Teoria dos Conjuntos. Belo Horizonte: UFMG, [20--]. Disponível em: https://homepages.dcc.ufmg.br/~loureiro/md/md_5TeoriaDosConjuntos.pdf. Acesso em: 16 maio 2026.

RAHM, Erhard; DO, Hong Hai. Data Cleaning: Problems and Current Approaches. IEEE Bulletin of the Technical Committee on Data Engineering, v. 23, n. 4, p. 3-13, 2000.

SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. Database System Concepts. 7. ed. [S. l.]: McGraw-Hill, 2019.

SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. Query Optimization. [S. l.: s. n.], [20--]. Disponível em: <https://www.db-book.com/slides-dir/PDF-dir/ch16.pdf>. Acesso em: 16 maio 2026.

STONEBRAKER, Michael; HELLERSTEIN, Joseph M. What Goes Around Comes Around. In: Readings in Database Systems. 4. ed. [S. l.]: MIT Press, 2005.

WINAND, Markus. SQL Performance Explained: everything developers need to know about SQL performance. [S. l.]: Markus Winand, 2012.